DEERFLOW 2.0 FROM ZERO TO HERO

A Practical Guide to the Open-Source SuperAgent Harness

JIALU CHENG

2026-04-22

DeerFlow 2.0 from Zero to Hero

作者： Jialu Cheng

版本： v1.0

日期： 2026-04-22

简介： From setting up DeerFlow from scratch to completing deep research, automated coding, content creation, and data analysis tasks

---

版权所有 © 2026 Jialu Cheng。保留所有权利。
未经书面许可，不得以任何形式复制或传播本书内容。

目录

DeerFlow 2.0 from Zero to Hero

Reading Guide

Part 1: Getting Started

§ 01 Why DeerFlow 2.0

§ 02 Installing DeerFlow: From Zero to First Run

§ 03 First Real Project: An Industry Research Report with DeerFlow

Part 2: Core Capabilities

§ 04 Skills System: What DeerFlow Can Do Depends on This

§ 05 Sub-Agents: One Person Can’t Finish, Call More

§ 06 Sandbox: DeerFlow’s “Real Computer”

Part 3: Advanced Practice

§ 07 IM Integration: Connect DeerFlow to WeChat, Lark, Telegram

§ 08 Model Configuration and Performance Tuning

§ 09 Production Deployment: From localhost to Team Usage

§ 10 From “Using AI” to “Directing AI”: Mindset Transformation

Appendix

DEERFLOW 2.0 FROM ZERO TO HERO

A Practical Guide to the Open-Source SuperAgent Harness

Creator: Jialu Cheng **For**: Developers, product managers, and technical leaders who want AI Agents to do real work **Based on**: DeerFlow 2.0 (bytedance/deer-flow, GitHub 63k+ Stars) **Last Updated**: 2026-04-22 **Use Cases**: Setting up DeerFlow from scratch to completing deep research, automated coding, content creation, and data analysis tasks

---

Reading Guide

DAY	CHAPTERS	GOAL
Day 1	§ 01- § 03	Understand what DeerFlow is, install it, and run your first task
Day 2-3	§ 04- § 06	Master the three core capabilities: Skills, Sub-Agents, and Sandbox
Day 4-5	§ 07- § 09	Advanced: IM integration, custom Skills, production deployment
Day 6	§ 10 + Appendix	Mindset transformation + Agent framework trend analysis

---

Part 1: Getting Started

From zero to one. After reading this, you’ll be able to run your first project.

---

§ 01 Why DeerFlow 2.0

01.1 How I Discovered DeerFlow

February 28, 2026. I was scrolling through GitHub Trending when I spotted a project called deer-flow. It was already #1 in stars.

I clicked on it. The README said "SuperAgent Harness." To be honest, I didn't think much of it at first. From 2025 to early 2026, there were countless Agent frameworks. LangGraph, CrewAI, AutoGen, OpenManus—a new one every two weeks.

But this one was different.

First, it came from ByteDance. Not a personal project, but something backed by an organization. Second, its positioning wasn't "yet another Agent framework." It called itself a "Harness"—an execution engine. Third, it gave Agents their own computer.

I spent three days getting it running. Then two more weeks diving deep into its architecture, Skills system, and Sandbox mechanism.

My conclusion: DeerFlow 2.0 is the most worthy open-source Agent project to learn in the first half of 2026. Period.

01.2 What Makes DeerFlow Different from Other Frameworks

Here's the conclusion first. Most Agent frameworks are "text generators." DeerFlow is a "task executor."

What's the difference? Ask ChatGPT to write Python code, and you get a code block. You copy, paste, run, and debug yourself. Ask DeerFlow to do the same thing, and it runs it in a Docker container and gives you the result directly.

Not simulated execution. Real execution.

DIMENSION	TRADITIONAL AGENT FRAMEWORK	DEERFLOW 2.0	MY TAKE
Code Execution	Generates code text	Runs in Docker container	This is the fundamental difference
Task Decomposition	Single-agent linear	Lead Agent + parallel Sub-Agents	Essential for complex tasks
File System	None or temporary	Persistent workspace + uploads + outputs	Enables long-running tasks
Memory	Within session only	Persistent across sessions	Gets better with use
Extensibility	Write code	Markdown-based SKILL.md	Even non-devs can write Skills
Model Support	Usually locked	Any OpenAI-compatible API	Key to saving money

> ***Key Point**: The first row— "Code Execution." This single line determines whether DeerFlow is an Agent platform or just an Agent framework.*

01.3 The History of DeerFlow

DeerFlow stands for **D**eep **E**xploration and **E**fficient **R**esearch **Flow**.

In 2025, it was a Deep Research tool inside ByteDance. It used LLMs + search + crawling for deep information research.

But when the community got the source code, they started "misusing" it—building data pipelines, generating PowerPoints, automating content production. The ByteDance team saw this and made a decision: a complete rewrite.

On February 27, 2026, DeerFlow 2.0 went open source. It shared zero code with v1.

It hit #1 on GitHub Trending within 24 hours. As of April 2026, it has 63,100+ Stars and 8,200+ Forks.

> ***My Experience**: How I judge the value of an open-source project*
>
> *I don't look at star growth speed. I look at three things: First, the quality of discussions in the Issues section. DeerFlow has 459 Issues and 214 PRs—very active discussions. Second, commit frequency. 1,938 commits means continuous iteration. Third, whether it has big company backing. ByteDance isn't a small team.*

01.4 Who It's For / Who It's Not For

It's for you if you are:

- A developer who wants AI Agents to actually do work for you
- A product manager who needs automated research reports and competitive analysis
- A technical leader who wants to deploy an AI Agent platform for your team
- A tech enthusiast curious about Agent execution engine internals

Not for you yet if you:

- Just want to call an API to generate text. ChatGPT is enough for that
- Need ultra-low latency real-time conversation. DeerFlow is a long-task engine
- Know nothing about Docker. You at least need to know `docker compose up`

Install it first, then decide. Next chapter, we start the installation.

---

§ 02 Installing DeerFlow: From Zero to First Run

02.1 What You Need

Environment Requirements:

TOOL	MINIMUM VERSION	PURPOSE
Docker	20.10+	Runs Sandbox containers
Node.js	22+	Frontend build
Python	3.11+	Backend runtime
pnpm	Latest	Frontend package management
uv	Latest	Python dependency management
Git	Any	Clone code

API Key (at least one):

SERVICE	PURPOSE	REQUIRED
OpenAI / DeepSeek / other LLM	Model calls	Yes (choose one)
Tavily	Web search	Recommended (optional)

Hardware:

- Minimum: 4-core CPU, 8GB RAM, 20GB disk
- Recommended: 8-core CPU, 16GB RAM

> ***Warning***: *Insufficient memory will cause crashes*
>
> *I first tried to run it on a 2-core 4GB cloud server. It OOM'd immediately. DeerFlow's Docker Sandbox + LangGraph Server + frontend consume significant resources. 8GB is the minimum.*

Estimated installation time: 10-15 minutes.

02.2 What We'll Achieve

After installation, you'll open `http://localhost:2026` in your browser and see the DeerFlow web interface. Type a question like "Help me research the top 5 open-source Agent frameworks in 2026," and DeerFlow will search the web, crawl pages, analyze data, and generate a cited research report.

This isn't ChatGPT-style Q&A. You'll see it execute step by step: plan first, then search, then analyze, then synthesize. The whole process takes 2-5 minutes. But the result is a document you can use directly.

02.3 Installation Steps

Step 1: Clone the repository

```
git clone https://github.com/bytedance/deer-flow.git
cd deer-flow
```

Expected result: You're in the deer-flow directory, with backend, frontend, docker folders inside.

Step 2: Run the configuration wizard

```
make setup
```

This launches an interactive wizard. It will ask you:

1. Which LLM Provider? (OpenAI / DeepSeek / Anthropic / …)
2. Configure search? (Recommended to set up Tavily)
3. Which Sandbox mode? (Docker recommended)

The wizard will auto-generate `config.yaml` and `.env` files. Takes about 2 minutes.

> ***My Experience***: *Choose DeepSeek for first-time setup*
>
> *I’ve tested several models. For the first experience, I recommend DeepSeek v3.2. Three reasons: First, it’s cheap. Second, it’s good at Chinese. Third, it’s officially recommended by DeerFlow. You can switch to GPT-4o or Claude after you get it running.*

Step 3: Verify configuration

```
make doctor
```

Expected result: All green √. If you see red ✗, fix according to the prompts.

Step 4: Choose startup method

Docker method (recommended):

```
make docker-init    # Pull Sandbox image (only once)
make docker-start   # Start all services
```

Local development method:

```
make check     # Check environment
make install   # Install dependencies
make dev       # Start development server
```

Expected result: Terminal shows services starting successfully.

Step 5: Open browser

Visit `http://localhost:2026`

Expected result: You see the DeerFlow web interface. A chat input box with a settings panel on the left.

> ***Warning**: Port 2026 occupied?*
>
> *If port 2026 is taken, check if a previous service didn't shut down. Run `make stop` or `make docker-stop` first.*

02.4 First Conversation

Type in the input box:

```
Help me research the DeerFlow project's basic information
```

Click send. Then watch.

You'll see DeerFlow do these things:

1. **Plan**: Lead Agent analyzes your question and creates a research plan
2. **Search**: Calls Tavily search engine to find relevant information
3. **Crawl**: Uses Web Fetch to grab content from key pages
4. **Analyze**: Synthesizes multiple sources, extracting key data
5. **Output**: Generates a structured research report

You can see real-time progress in the interface.

> ***My Experience**: First run is slow*
>
> *My first run took about 4 minutes. Be patient. DeerFlow isn't an instant-response chatbot. It's an execution engine that takes time to do real work. If it takes more than 10 minutes, your API Key is probably misconfigured or there's a network issue.*

02.5 Recap

Here, we did 5 things:

1. Cloned DeerFlow code
2. Generated configuration via wizard
3. Started Docker services

4. Opened the web interface
5. Completed the first deep research

Took about 15 minutes. You now have a running DeerFlow instance.

> ***Core Advice**: Get it running first, then tweak configuration*
>
> *Many people dive into config.yaml and study every parameter first. Don't. Use default config to get it running first. Once you feel its capabilities, you'll naturally know what to tweak.*

Installation done, first run successful. Next chapter, we do a real project.

---

§03 First Real Project: An Industry Research Report with DeerFlow

03.1 Project Requirements

Scenario: You're a product manager. Your boss wants an "AI Agent Framework Market Analysis Report for 2026" on their desk by tomorrow morning.

Previously, you would: Open Google, search 10 pages, read them one by one, take notes, compile into a document. Takes at least 3-4 hours.

Now, you'll use DeerFlow to get it done in 20 minutes.

03.2 Building from Scratch

Open `http://localhost:2026`.

Type in the input box:

```
Please create a market analysis report on AI Agent frameworks in 2026. Requirements:
1. Cover at least 5 mainstream frameworks (LangGraph, CrewAI, AutoGen, DeerFlow, OpenManus, etc.)
2. For each framework include: core positioning, technical architecture, star count, use cases
3. Create a comparison table
4. Give me your selection recommendations
5. Output as a structured Markdown report
```

Click send. Then go get a coffee.

When you come back, you’ll see DeerFlow did this:

Phase 1: Planning. Lead Agent broke your requirement into sub-tasks:

- Sub-Agent 1: Research LangGraph
- Sub-Agent 2: Research CrewAI
- Sub-Agent 3: Research AutoGen
- Sub-Agent 4: Research DeerFlow itself
- Sub-Agent 5: Research OpenManus
- Sub-Agent 6: Synthesize comparison + recommendations

Phase 2: Parallel Execution. 5 research Sub-Agents started simultaneously. Each Agent independently searched, crawled, and analyzed. No interference.

Phase 3: Synthesis. Lead Agent collected all Sub-Agent results and generated the final report.

Phase 4: Output. A complete Markdown report appeared in the chat window.

> ***My Experience**: The Power of Sub-Agents*
>
> *I did a similar framework comparison study with DeerFlow. Single-agent linear execution took about 15 minutes. Sub-Agents parallel execution took 5 minutes. Similar quality, 3x faster. That’s the practical value of multi-agent orchestration.*

03.3 Result Analysis

DeerFlow’s output typically includes:

- A summary paragraph for each framework (3-5 sentences)
- Key data (Stars, Forks, License)

- Comparison table
- Selection recommendations

How’s the quality?

Honestly, the first generated report scores about 70/100. Data is mostly accurate. Structure is clear. But depth is lacking—some parts feel like search result stitching.

How to improve? Two methods:

Method 1: Follow-up questions. In the same conversation, continue typing:

```
Add "learning curve" and "community activity" dimensions to the comparison table.
Also, the LangGraph section is too short—expand on its state management mechanism.
```

DeerFlow will supplement and modify based on existing research, without starting from scratch.

Method 2: Switch models. If you used DeepSeek first, try GPT-4o or Claude. Different models show obvious differences in analysis depth and writing quality.

Change model config in `config.yaml`:

```yaml
models:
  - name: gpt-4o
    display_name: GPT-4o
    use: langchain_openai:ChatOpenAI
    model: gpt-4o
    api_key: $OPENAI_API_KEY
```

Restart services to apply.

03.4 Pitfalls

Pitfall 1: Insufficient search results

If you didn’t configure a Tavily API Key, DeerFlow’s search capability is limited. It might only use built-in simple search, resulting in lower quality results.

Solution: Register at `tavily.com` to get an API Key. Free tier: 1,000 searches/month, enough for use. Add to `.env`:

```
TAVILY_API_KEY=tvly-your-key-here
```

Pitfall 2: Report too long causing truncation

DeerFlow sometimes generates very long reports. If you see the output cut off, it's probably the model's max tokens limit.

Solution: Increase max_tokens in config.yaml, or add "keep report under 3,000 words" to your prompt.

Pitfall 3: Sub-Agent task failure

Occasionally one Sub-Agent fails to search or times out. Lead Agent skips that Agent's result and continues synthesis.

You'll notice the report is missing analysis for one framework. Just ask again.

> ***Core Advice**: DeerFlow isn't a one-shot perfect tool*
>
> *It's an execution engine. First attempt might only hit 70%. But you can continuously optimize in the same session. After 2-3 rounds of interaction, you can reach 85-90%. Much faster than writing from scratch.*

03.5 Recap

In this chapter, we did something real: used DeerFlow to generate an industry research report.

Key takeaways:

1. DeerFlow's Sub-Agents can research multiple targets in parallel
2. First result may be imperfect—follow-up questions significantly improve quality
3. Search API Key has big impact on result quality
4. Switching models improves analysis depth

Installed, running, first project done. Next chapter starts diving into DeerFlow's core capabilities. Part 2, let's talk about the Skills system.

Part 2: Core Capabilities

Deep dive into DeerFlow's three core mechanisms. One per chapter. After reading, you'll be able to use 80% of DeerFlow's features.

---

§ 04 Skills System: What DeerFlow Can Do Depends on This

04.1 Skills Aren't Plugins, They're "Work Guides"

First, correct a common misconception.

Many people see DeerFlow's Skills and think they're like VS Code plugins—a code package that adds a feature when installed.

No.

A DeerFlow Skill is a Markdown file. Yes, a `.md` file. It tells the Agent in natural language: how to do this type of task, what tools to use, what format to output.

My analogy:

- **Traditional plugins** are like installing a part in software. The Agent doesn't know it exists; you manually call it
- **DeerFlow's Skill** is like writing an SOP for the Agent. The Agent reads the SOP and knows when and how to use it

This design is clever. Three reasons:

First, **extremely low barrier**. Can write Markdown? Can write Skills. No Python needed, no need to understand Agent framework APIs.

Second, **progressive loading**. DeerFlow doesn't dump all Skills into context at once. It dynamically loads relevant Skills based on task needs. This allows it to work with token-limited models.

Third, **composable**. Multiple Skills can work together. A research task might use `research`, `report-generation`, and `slide-creation` Skills simultaneously.

04.2 Built-in Skills Overview

DeerFlow 2.0 comes with these Skills out of the box:

SKILL NAME	PATH	FUNCTION
research	/mnt/skills/public/research/	Deep web research
report-generation	/mnt/skills/public/report-generation/	Structured report generation
slide-creation	/mnt/skills/public/slide-creation/	PowerPoint slide creation
web-page	/mnt/skills/public/web-page/	Web page generation
image-generation	/mnt/skills/public/image-generation/	Image generation
claude-to-deerflow	/mnt/skills/public/claude-to-deerflow/	Call DeerFlow from Claude Code

> ***My Experience**: Most used are research + report-generation*
>
> *In two months of using DeerFlow, 80% of my time was with these two Skills. Deep research + report generation covers most knowledge worker needs. slide-creation is great for presentations to leadership.*

04.3 Write Your First Custom Skill

This is one of DeerFlow’s most valuable capabilities. Let me walk you through writing one.

Scenario: You frequently do competitive analysis. Each time you repeat telling DeerFlow the analysis framework and output format. Let’s solidify this into a Skill.

Step 1: Create directory

```
mkdir -p skills/custom/competitive-analysis
```

Step 2: Write SKILL.md

```
---
name: competitive-analysis
description: Deep competitive analysis skill
version: 1.0.0
author: your-name
---

# Competitive Analysis Skill

## Task Definition
Use this skill when the user asks to analyze competitors of a product.

## Execution Steps
1. Identify the target product mentioned by the user
2. Search for top 5 competitors in that product's category
3. For each competitor, collect:
   - Product positioning (one sentence)
   - Core features (3-5 items)
   - Pricing model
   - User scale (if public data available)
   - Strengths and weaknesses
4. Generate comparison table
5. Provide selection recommendations

## Output Format
- Markdown format
- Must include comparison table
- Table must have "My Conclusion" column (you can change to your name)
- Final paragraph is "Selection Recommendation"

## Tool Usage
- Use web_search to search for competitor information
- Use web_fetch to get product details from official websites
- Use Python scripts for data analysis if needed
```

Step 3: Put in the right location

Put `competitive-analysis/SKILL.md` in `/mnt/skills/custom/` directory.

If using Docker, this directory is mounted into the container. If using local development, put it in the project's `skills/custom/` folder.

Step 4: Test

After restarting DeerFlow, type:

```
Do a competitive analysis for Notion
```

DeerFlow’s Lead Agent will automatically detect this is a competitive analysis task, load your newly written Skill, and execute according to the defined steps.

> ***Core Advice**: Skills are essentially “reusable prompt engineering”*
>
> *You solidify a good prompt template into a file, and DeerFlow can reuse it repeatedly. Much more efficient than writing long prompts in chat every time.*

04.4 Advanced Skill Usage

Skill with Scripts:

SKILL.md can reference script files in the same directory. For example, your competitive analysis needs a fixed data cleaning script:

```
skills/custom/competitive-analysis/
├── SKILL.md
└── scripts/
    └── clean_data.py
```

Reference in SKILL.md:

```
## Data Cleaning
Use scripts/clean_data.py to clean raw data
```

Agent will run this script in the Sandbox during execution.

Install External Skill via Gateway:

DeerFlow supports `.skill` package format. Community developers can package and share Skills. You install via Gateway API:

```
curl -X POST http://localhost:2026/api/skills/install \
  -F "file=@my-skill.skill"
```

Automatically available after installation.

That’s DeerFlow’s Skills system. It determines what DeerFlow can do. Next chapter, let’s talk about Sub-Agents—how DeerFlow does multiple things at once.

---

§ 05 Sub-Agents: One Person Can’t Finish, Call More

05.1 Why Sub-Agents Are Needed

Imagine this scenario.

You ask an Agent to do this: “Help me research AI Agent frameworks, compare 5 products, then generate a PowerPoint.”

If there’s only one Agent, it executes serially: search first product, then second, then third… after all searches, start comparing… after comparison, start making PPT. Whole process takes 15-20 minutes.

DeerFlow does it differently. Lead Agent receives the task and breaks it down:

```
Lead Agent plans:
├── Sub-Agent A → Research LangGraph
├── Sub-Agent B → Research CrewAI
├── Sub-Agent C → Research AutoGen
├── Sub-Agent D → Research DeerFlow
├── Sub-Agent E → Research OpenManus
└── Synthesis → Comparison analysis + Generate PPT
```

5 research Agents run in parallel. Each has its own context, tools, and termination conditions. After completion, Lead Agent collects results and makes the PPT.

Time compressed from 20 minutes to 5 minutes. Quality unchanged.

05.2 How Lead Agent Decomposes Tasks

There’s a key mechanism here. Lead Agent doesn’t randomly break tasks. It has task decomposition logic:

1. **Identify independent sub-tasks**: What can be done in parallel? (e.g., researching 5 products doesn’t depend on each other)

2. **Identify dependencies**: What must wait for prerequisites? (e.g., comparison analysis must wait for all research to complete)
3. **Assign tools and Skills**: What tools does each Sub-Agent need? (Research Agent needs search+fetch, PPT Agent needs slide-creation)
4. **Set termination conditions**: When is each Sub-Agent complete?

> ***My Experience**: Model capability determines decomposition quality*
>
> *I tested different models' task decomposition effects. GPT-4o and Claude have the most reasonable decomposition, with clear sub-task boundaries. DeepSeek v3.2 is also good. But small models (7B level) often decompose poorly—either too granular or parallel tasks become serial. If Sub-Agents frequently fail, check if Lead Agent's model is too weak.*

05.3 Context Isolation

This is my favorite part of DeerFlow's design.

Each Sub-Agent runs in **completely isolated context**. Sub-Agent A sees nothing from Sub-Agent B. Lead Agent also doesn't dump all Sub-Agents' intermediate processes into its own context.

Why does this matter?

First, **avoid context contamination**. If 5 Agents share context, one Agent's noise might affect others' judgment.

Second, **save tokens**. Each Agent only processes information relevant to its task. No paying for irrelevant content.

Third, **fault isolation**. One Sub-Agent crashes, doesn't affect others. Lead Agent skips the failed Agent and continues execution.

05.4 Practice: Multi-dimensional Research with Sub-Agents

Try this prompt:

```
Please comprehensively analyze the Notion product:
1. Product features and positioning
2. Business model and pricing
3. Competitor comparison (at least 3 competitors)
4. User sentiment analysis (collect from Reddit and Twitter)
5. Generate a complete analysis report
```

Watch DeerFlow’s execution process. You’ll see:

- Sub-Agent 1: Searches Notion’s product features and latest updates
- Sub-Agent 2: Finds Notion’s pricing page and business model
- Sub-Agent 3: Searches and analyzes competitors (Obsidian, Coda, Slite, etc.)
- Sub-Agent 4: Searches user reviews on Reddit and Twitter
- Final synthesis: Lead Agent merges 4 dimensions into one report

4 research tasks running simultaneously, then synthesized. This is typical Sub-Agents usage.

> ***Core Advice**: Write sub-tasks clearly in the prompt*
>
> *DeerFlow’s Lead Agent can decompose tasks itself. But if you explicitly list sub-tasks in the prompt (like my numbered list above), it decomposes more accurately. Less room for AI guessing means higher quality.*

Sub-Agents solve “how to do multiple things at once.” Next chapter, we enter DeerFlow’s most hardcore part—the Sandbox.

---

§06 Sandbox: DeerFlow’s “Real Computer”

06.1 Most Agents Don’t Have Hands

Here’s a counter-intuitive fact.

In 2026, the vast majority of AI Agents still “don’t have hands.” They can think, they can speak, but they can’t do. Ask them to write code, you get a code block. Ask them to analyze data, you get analysis ideas. Real execution? Still on you.

DeerFlow gave Agents hands. Specifically, a Docker virtual machine.

Each task runs in an isolated Docker container. The container has:

- Complete Linux file system
- Bash terminal
- Python runtime
- Network access capability
- Persistent storage

```
/mnt/user-data/
├── uploads/     ← Files you upload
├── workspace/   ← Agent's working directory
└── outputs/     ← Final deliverables
```

06.2 Three Sandbox Modes

DeerFlow supports 3 execution modes:

MODE	ISOLATION LEVEL	USE CASE	MY TAKE
Local Execution	No isolation	Local dev/testing	Convenient but dangerous
Docker Execution	Container isolation	Recommended daily use	Balance of safety and performance
Docker + Kubernetes	Pod isolation	Multi-user/enterprise deployment	Good for team sharing

> ***Warning**: In Local mode, Bash is disabled by default*
>
> *DeerFlow disables Host Bash in local mode. Because without isolation, Agent-executed commands run directly on your machine. If you're sure it's a trusted scenario, you can manually enable in config.yaml. But I don't recommend this for production environments.*

06.3 What Can Be Done in Sandbox

Data Analysis:

Upload a CSV file, tell DeerFlow:

```
Help me analyze this dataset, find key factors affecting sales, generate visualization charts
```

DeerFlow in Sandbox will:

1. Use Python to read your CSV
2. Install needed packages (pandas, matplotlib, etc.)
3. Write analysis script
4. Execute script
5. Generate chart files
6. Put charts in outputs directory

You just download the images from outputs.

Code Projects:

```
Help me build a Todo app scaffold with React+FastAPI
```

DeerFlow in workspace will:

1. Create project directory structure
2. Write frontend code
3. Write backend code
4. Install dependencies
5. Run tests
6. Package entire project to outputs

Web Page Generation:

```
Help me create a Landing Page showcasing DeerFlow features
```

DeerFlow uses web-page Skill, writes HTML/CSS/JS in Sandbox, directly outputs a usable web page file.

> ***My Experience**: Sandbox’s real value is “verifiable”*
>
> *Previously, asking AI to write code meant I had to verify if it ran. Now DeerFlow verifies itself. It runs the code after writing. If it doesn’t run, it fixes itself. This closed loop saves me tons of debug time.*

06.4 File System Persistence

Sandbox’s file system isn’t disposable.

`/mnt/user-data/workspace/` is persistent. Files generated in the last task are still available in the next task. This allows DeerFlow to do long-term projects, not just one-off tasks.

For example, on day 1 you had DeerFlow scaffold a project. On day 2 you can say:

```
Continue yesterday's project, add a user login feature
```

DeerFlow continues development based on the previous workspace.

06.5 Security Boundaries

Sandbox is the core of DeerFlow’s security model. Key points:

1. **Only listens on localhost by default**. DeerFlow is designed for local use. If you want public deployment, you must add your own authentication gateway
2. **Docker isolation**. Code in Sandbox cannot access host machine’s other processes
3. **Output review**. Gateway layer forcibly sets HTML/SVG and other active content types to “download” rather than “inline render,” preventing XSS
4. **Network controllable**. You can limit container’s network egress in Docker configuration

> ***Core Advice**: Don’t expose DeerFlow to the public internet naked*
>
> *I’ve seen people deploy DeerFlow directly on cloud servers with ports open to the world, no authentication. This is equivalent to giving anyone a remote shell that can execute arbitrary code. Either add an authentication gateway (Nginx + Basic Auth), or add IP whitelisting.*

Sandbox is DeerFlow’s execution engine. With it, Agents go from “talkers” to “doers.”

Here, Part 2’s three core capabilities are all covered. Skills determine what Agents can do. Sub-Agents let Agents do multiple things at once. Sandbox lets Agents actually get their hands dirty.

Next chapter starts Part 3, we enter advanced practice.

Part 3: Advanced Practice

More scenarios. Deeper configuration. Usage closer to production.

---

§07 IM Integration: Connect DeerFlow to WeChat, Lark, Telegram

07.1 Why Connect IM

Isn’t the web interface enough?

Often not.

Your boss won’t open localhost:2026 to send tasks to DeerFlow. They’ll say in the WeChat group “Who can check the XX data for me?” Your team discussions won’t move to DeerFlow’s interface. They’re in Lark group chats.

DeerFlow 2.0 has 5 IM channels built-in. After configuration, you can talk to DeerFlow directly on these platforms. It's like a "digital employee" in the group.

07.2 Supported Platforms

PLATFORM	TRANSPORT	CONFIG DIFFICULTY	NEEDS PUBLIC IP
Telegram	Bot API long polling	Easy	No
Slack	Socket Mode	Medium	No
Lark	WebSocket	Medium	No
WeChat	Tencent iLink long polling	Medium	No
WeCom	WebSocket	Medium	No

> ***Key Point**: None need public IP.*
>
> *DeerFlow uses long polling or WebSocket, actively connecting to platform servers from your machine. No need to configure callback URLs, no need to be publicly reachable. Very friendly for local deployment.*

07.3 Practice: Connect to Telegram

Telegram is the easiest to configure. 5 minutes and done.

Step 1: Create Bot

Open Telegram, find @BotFather, send `/newbot`. Follow prompts to set Bot name. You'll get an HTTP API Token, looks like:

```
123456789:ABCdefGHIjklMNOpqrSTUvwxYZ
```

Step 2: Configure DeerFlow

Add to `.env`:

```
TELEGRAM_BOT_TOKEN=your_token
```

Add to `config.yaml`:

```yaml
channels:
  telegram:
    enabled: true
    bot_token: $TELEGRAM_BOT_TOKEN
    allowed_users: []  # Empty means allow everyone
```

Step 3: Restart DeerFlow

```
make docker-stop && make docker-start
```

Step 4: Chat with Bot in Telegram

Find your created Bot, send a message:

```
Help me search today's AI news
```

Bot will reply with DeerFlow’s execution results.

> ***My Experience**: Limit allowed_users*
>
> *If others discover your Bot, they can use it too. Every message calls your LLM API, costs money. Add your Telegram user ID to config.yaml to restrict access.*

07.4 IM Channel Commands

When chatting with DeerFlow in IM, besides normal chat, there are these commands:

COMMAND	FUNCTION
`/new`	Start new conversation
`/status`	Check current thread status
`/models`	List available models
`/memory`	View memory
`/help`	Help

Messages without command prefix are treated as normal chat.

IM integration transforms DeerFlow from a "developer tool" to a "team tool." Next chapter, we dive deeper into configuration.

---

§ 08 Model Configuration and Performance Tuning

08.1 Core Principles of Model Selection

DeerFlow is model-agnostic. As long as it's OpenAI-compatible API, it works. But "works" and "works well" are very different.

I've tested over a dozen model combinations. Core conclusion:

USE CASE	RECOMMENDED MODEL	REASON	MY TAKE
Lead Agent (task decomposition)	GPT-4o / Claude Sonnet 4	Needs strong logic and structured output	Don’t save money here
Research Sub-Agent	DeepSeek v3.2	Best cost-performance, good at Chinese	Economies of scale
Code generation	Claude Sonnet 4 / GPT-4o	Code quality determines Sandbox execution success	Bad code wastes more tokens retrying
Simple text tasks	DeepSeek v3.2 / Kimi 2.5	Cheap and sufficient	Save where you can

> ***My Experience**: Lead Agent’s model is most important*
>
> *I did an experiment. Lead Agent with GPT-4o, Sub-Agents with DeepSeek. vs. both with DeepSeek. The former had ~20% higher task completion rate. Reason: Lead Agent handles decomposition and synthesis, both requiring strongest model. Sub-Agents do specific execution—slightly weaker model doesn’t matter much.*

08.2 Multi-Model Mixed Configuration

DeerFlow supports configuring multiple models in config.yaml:

```yaml
models:
  - name: gpt-4o
    display_name: GPT-4o (Lead Agent)
    use: langchain_openai:ChatOpenAI
    model: gpt-4o
    api_key: $OPENAI_API_KEY

  - name: deepseek-v3
    display_name: DeepSeek v3.2 (Sub-Agents)
    use: langchain_openai:ChatOpenAI
    model: deepseek-chat
    api_key: $DEEPSEEK_API_KEY
    base_url: https://api.deepseek.com/v1

  - name: qwen3-local
    display_name: Qwen3 32B (Local)
    use: deerflow.models.vllm_provider:VllmChatModel
    model: Qwen/Qwen3-32B
    api_key: $VLLM_API_KEY
    base_url: http://localhost:8000/v1
```

Switch models in the top right of the web interface. Lead Agent and Sub-Agents can use different models.

08.3 Local Model Deployment

Want to save API costs? Can use local models. DeerFlow supports deploying local models via Ollama or vLLM.

Ollama way (simplest):

```bash
# After installing Ollama
ollama pull qwen3:32b
```

In config.yaml:

```yaml
models:
  - name: qwen3-ollama
    display_name: Qwen3 32B (Ollama)
    use: langchain_openai:ChatOpenAI
    model: qwen3:32b
    base_url: http://localhost:11434/v1
    api_key: ollama  # placeholder
```

> ***Warning**: Limitations of local models*
>
> *Local 7B-14B models are okay for Sub-Agent execution, but not great for Lead Agent task decomposition. Recommend at least 32B level models for Lead Agent. If your machine can't handle 32B, use cloud API for Lead Agent and local models for Sub-Agents.*

08.4 Gateway Mode vs. Standard Mode

DeerFlow has two runtime modes:

DIMENSION	STANDARD MODE	GATEWAY MODE
Architecture	Gateway + LangGraph separated	Gateway embedded in Agent runtime
Processes	4	3
Resource consumption	Higher	Lower
Cold start	Slower	Faster
License	Needs LangGraph Platform License (production)	Not needed

My recommendation:

- Personal use / small team → Gateway mode (`make dev-pro`)
- Enterprise deployment / needs LangGraph Studio → Standard mode (`make dev`)

Gateway mode is experimental, but I've used it for a month without issues. About 30% less resource consumption.

This chapter covered model and runtime mode configuration. Next chapter does something more complex: production deployment.

§ 09 Production Deployment: From localhost to Team Usage

09.1 From Toy to Tool

In the previous 8 chapters, DeerFlow ran on your localhost. One person using.

This chapter, we deploy it as a service the whole team can use.

09.2 Deployment Architecture

Recommended architecture for production deployment:

```
User → Nginx(reverse proxy+auth) → DeerFlow Gateway → LangGraph Server
                                        ↓
                                  Docker Sandbox Pool
```

Key components:

1. **Nginx**: Reverse proxy + SSL + Basic Auth
2. **DeerFlow Gateway**: API service
3. **LangGraph Server**: Agent runtime
4. **Sandbox Pool**: Docker container pool

09.3 Docker Production Deployment

Step 1: Build images

```
make up  # Build and start all production services
```

Or step by step:

```
deploy.sh build        # Build images only
deploy.sh start        # Standard mode start
deploy.sh start --gateway  # Gateway mode start
```

Step 2: Add authentication

Add Basic Auth in Nginx config:

```
server {
    listen 443 ssl;
    server_name deerflow.your-domain.com;

    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    auth_basic "DeerFlow Access";
    auth_basic_user_file /etc/nginx/.htpasswd;

    location / {
        proxy_pass http://localhost:2026;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Generate password file:

```
htpasswd -c /etc/nginx/.htpasswd your-username
```

Step 3: Configure IP whitelist (optional but recommended)

```
allow 10.0.0.0/8;       # Internal network
allow your_public_ip;
deny all;
```

> ***Core Advice**: Security first*
>
> *I'll say it again. DeerFlow can execute arbitrary code. No authentication equals giving the world a remote shell. SSL + Basic Auth is the minimum. Enterprise environments should connect to SSO.*

09.4 Resource Planning

Choose configuration based on team size:

TEAM SIZE	CPU	MEMORY	DISK	CONCURRENT TASKS
1-3 people	8-core	16GB	40GB	2-3
5-10 people	16-core	32GB	100GB	5-8
10+ people	32-core+	64GB+	200GB+	Consider K8s

> ***My Experience**: Memory matters more than CPU*
>
> *DeerFlow's bottleneck is usually not CPU, but memory. Each Docker Sandbox container consumes at least 1-2GB. If running 5 tasks simultaneously, that's 10GB just for Sandboxes. Plus Gateway and LangGraph Server themselves, 32GB is the minimum for a 10-person team.*

09.5 Observability

Production environments must have monitoring. DeerFlow supports two options:

LangSmith (recommended):

Add to `.env`:

```
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2_pt_your_key
LANGSMITH_PROJECT=deerflow-prod
```

Can see each Agent call's trace, duration, token consumption on LangSmith Dashboard.

Langfuse (open source alternative):

```
LANGFUSE_TRACING=true
LANGFUSE_PUBLIC_KEY=pk-lf-your_key
LANGFUSE_SECRET_KEY=sk-lf-your_key
```

Both can be enabled simultaneously.

Deployment done. Final chapter, let's talk about mindset transformation.

§10 From "Using AI" to "Directing AI": Mindset Transformation

10.1 This Isn't ChatGPT

Throughout writing this book, I noticed an interesting phenomenon.

Many first-time DeerFlow users interact with it like ChatGPT: ask a question, wait for an answer. Not satisfied? Rephrase and ask again.

This is completely wrong.

DeerFlow isn't a Q&A bot. It's a command center for an Agent team. You're not a "questioner," you're a "project manager."

10.2 Three Mindset Transformations

Transformation 1: From "Q&A" to "Task Assignment"

ChatGPT style: `What are some AI Agent frameworks?`

DeerFlow style:

```
Help me complete the following task:
1. Search for top 10 open-source AI Agent frameworks in 2026
2. For each framework collect: star count, language, core features, latest update date
3. Sort by star count
4. Generate comparison table
5. Finally give your Top 3 recommendations with reasons
```

What's the difference? The second approach defines clear input, steps, and output. DeerFlow's Lead Agent can more accurately decompose tasks.

Transformation 2: From "One-shot Perfect" to "Iterative Optimization"

Don't expect DeerFlow to give you perfect results on the first try. It's designed to be iterative.

First attempt: 70-point result. Follow-up: “Add learning curve dimension to comparison table” → 80 points. Another follow-up: “LangGraph section too shallow, expand on state management” → 90 points.

3 rounds of interaction, 5 minutes, from 70 to 90 points. Much faster than writing a 90-point report from scratch yourself.

Transformation 3: From “Instant Response” to “Wait for Execution”

DeerFlow doing a deep research task might take 3-10 minutes. During this time, you should do something else, not stare at the screen waiting.

Most efficient usage: Open 3 DeerFlow threads simultaneously, give 3 different tasks. Let them run in parallel. Come back to collect results.

10.3 The Agent World After DeerFlow

April 2026, Agent technology is still early. But the direction is already clear.

I believe Agent technology will go through 3 stages:

STAGE	CHARACTERISTICS	REPRESENTATIVE PRODUCTS	TIMELINE
Stage 1: Assistant	AI answers questions	ChatGPT, Claude	2023-2025
Stage 2: Executor	AI executes tasks	DeerFlow, Claude Code	2025-2027
Stage 3: Collaborator	AI teams autonomously collaborate	Not yet appeared	2027+

DeerFlow stands at the forefront of Stage 2. It’s not the end, but it’s one of the best starting points.

> ***My Experience***: *Technology becomes obsolete, mindset doesn't*
>
> *DeerFlow might be replaced by better products in 2 years. But the "directing AI" mindset you develop while using DeerFlow is universal. Task decomposition, parallel execution, iterative optimization, persistent memory—these concepts will run through the entire Agent era.*

10.4 One Last Thing

I spent about 2 weeks writing this book. During that time, DeerFlow helped me complete a large amount of research and data collection work.

This isn't an ad. It's a fact.

DeerFlow 2.0 from Zero to Hero—from the installation in Chapter 1, to the mindset transformation now, you've completed the entire journey.

Put it to use.

Appendix

A 2026 Agent Framework Data Quick Reference

FRAMEWORK	STARS (APR 2026)	LICENSE	CORE LANGUAGE	POSITIONING
DeerFlow 2.0	63.1k	MIT	Python+TS	SuperAgent Harness
LangGraph	12k+	MIT	Python	State machine Agent orchestration
CrewAI	28k+	MIT	Python	Role-playing multi-agent
AutoGen	40k+	MIT	Python	Conversational multi-agent
OpenManus	15k+	Apache 2.0	Python	General-purpose Agent framework

Data source: GitHub, April 2026. Specific numbers may change over time.

B DeerFlow Common Commands Quick Reference

COMMAND	FUNCTION
`make setup`	Interactive configuration wizard
`make doctor`	Verify configuration
`make dev`	Local development start
`make dev-pro`	Local development + Gateway mode
`make docker-init`	Pull Sandbox image
`make docker-start`	Docker development start
`make up`	Production deployment
`make down`	Stop production services
`make stop`	Stop local services
`make check`	Check environment dependencies
`make install`	Install dependencies
`make config`	Copy full config template

C config.yaml Configuration Quick Reference

```yaml
# Model configuration
models:
  - name: model-name
    display_name: Display Name
    use: langchain_openai:ChatOpenAI
    model: model-id
    api_key: $ENV_VAR_NAME
    base_url: https://api.example.com/v1  # Optional

# Sandbox configuration
sandbox:
  use: deerflow.community.aio_sandbox:AioSandboxProvider
  # Or deerflow.sandbox:LocalSandboxProvider

# Search configuration (set TAVILY_API_KEY in .env)

# IM channel configuration
channels:
  telegram:
    enabled: true
    bot_token: $TELEGRAM_BOT_TOKEN

# Observability (set in .env)
# LANGSMITH_TRACING=true
# LANGSMITH_API_KEY=your-key
```

Full Reading Guide

TIME	CHAPTER	GOAL	EST. READING TIME
Day 1 AM	§ 01 Why DeerFlow	Understand positioning and value	15 min
Day 1 PM	§ 02 Installation	Get first run working	30 min (incl. hands-on)
Day 1 Evening	§ 03 First Project	Complete a research report	30 min (incl. hands-on)
Day 2	§ 04 Skills System	Understand and write first custom Skill	45 min
Day 3	§ 05 Sub-Agents	Understand multi-agent orchestration	30 min
Day 3	§ 06 Sandbox	Understand execution engine	30 min
Day 4	§ 07 IM Integration	Connect to an IM platform	45 min (incl. hands-on)
Day 4	§ 08 Model Tuning	Configure multiple models	30 min
Day 5	§ 09 Production Deployment	Deploy for team use	60 min (incl. hands-on)
Day 6	§ 10 Mindset Transformation	Build “directing AI” mental model	20 min

---

> *Created by Jialu Cheng | AI Native Coder · Independent Developer*
> *X: Jialu Cheng (@Xialu8888)*

DEERFLOW 2.0 FROM ZERO TO HERO

From setting up DeerFlow from scratch to completing deep research, automated coding, content creation, and data analysis tasks